CS3500 Operating Systems
Page Tables Lab

Due: 11:59 PM, Sunday, 20th September 2026

1 Background Reading & Environment Setup

1.1 Introduction

In this lab you will explore page tables and modify them to implement common OS features.

1.2 Background Reading

Before you start coding, read Chapter 3 of the xv6 book, and the related files:

- `kernel/memlayout.h`
- `kernel/vm.c`
- `kernel/kalloc.c`

It may also help to consult the RISC-V privileged architecture manual.

1.3 Setting Up the Lab

Clone the repository:

https://github.com/Aryakrish05/CS3500-Page-Tables-Lab

The docker container used for the previous labs may be used here as well.

```
$ git clone https://github.com/Aryakrish05/CS3500-Page-Tables-Lab

$ docker run -it -v <path-to-cloned-repo>:/home/xv6-labs nandhagk/xv6-
    tools:latest
```

This starts the Docker container with the `pgtbl` lab mounted and ready to use.

2 Inspect a user-process page table

To help you understand RISC-V page tables, your first task is to explain the page table for a user process.
Run `make qemu` and run the user program `pgtbltest`. The `print_pgtbl` functions prints out the page-table entries for the first 10 and last 10 pages of the `pgtbltest` process using the `pgpte` system call that we added to xv6 for this lab.
The output looks as follows:

```
va 0 pte 0x21FCF45B pa 0x87F3D000 perm 0x5B
va 1000 pte 0x21FCE85B pa 0x87F3A000 perm 0x5B
...
va 0xFFFFD000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFFE000 pte 0x21FD80C7 pa 0x87F60000 perm 0xC7
va 0xFFFFF000 pte 0x20001C4B pa 0x80007000 perm 0x4B
```

For every page table entry in the `print_pgtbl` output, explain what it logically contains, what its permission bits are, and why it has been mapped into the process's address space. Figure 3.4 in the xv6 book may be helpful, although note that the figure may contain a slightly different set of pages than the process being inspected here. Also note that xv6 does not place consecutive virtual pages in consecutive physical memory locations.

3 Optimising System Calls

Some operating systems (e.g., Linux) speed up certain system calls by sharing data in a read-only region between userspace and the kernel. This eliminates the need for kernel crossings when performing these system calls. To help you learn how to insert mappings into a page table, your task is to implement this optimization for the `getpid()` system call in xv6.
When each process is created, map one read-only page at USYSCALL (a virtual address defined in `memlayout.h`). At the start of this page, store a `struct usyscall` (also defined in `memlayout.h`), and initialize it to store the PID of the current process. For this lab, `ugetpid()` has been provided on the userspace side and will automatically use the USYSCALL mapping. A correct implementation for this part of the lab should pass the `ugetpid` test case when running `pgtbltest`.

Hints

- Choose permission bits that allow userspace to only read the page.
- There are a few things that need to be done over the lifecycle of a new page. For inspiration, understand the trapframe handling in `kernel/proc.c`.

Which other xv6 system call(s) could be made faster using this shared page? Explain how.

4 Print a page table

To help you visualize RISC-V page tables, and perhaps to aid future debugging, your next task is to write a function that prints the contents of a page table.
We added a system call `kpgtbl()`, which calls `vmprint()` in `vm.c`. It takes a `pagetable_t` argument, and your job is to print that pagetable in the format described below.
When you run `print_kpgtbl()` test, your implementation should print the following output:

```
page table 0x0000000087f22000
..0x0000000000000000: pte 0x0000000021fc7801 pa 0x0000000087f1e000
.. ..0x0000000000000000: pte 0x0000000021fc7401 pa 0x0000000087f1d000
.. .. ..0x0000000000000000: pte 0x0000000021fc7c5b pa 0x0000000087f1f000
.. .. ..0x0000000000001000: pte 0x0000000021fc705b pa 0x0000000087f1c000
.. .. ..0x0000000000002000: pte 0x0000000021fc6cd7 pa 0x0000000087f1b000
.. .. ..0x0000000000003000: pte 0x0000000021fc6807 pa 0x0000000087f1a000
.. .. ..0x0000000000004000: pte 0x0000000021fc64d7 pa 0x0000000087f19000
..0x0000003fc0000000: pte 0x0000000021fc8401 pa 0x0000000087f21000
.. ..0x0000003fffe00000: pte 0x0000000021fc8001 pa 0x0000000087f20000
.. .. ..0x0000003fffffd000: pte 0x0000000021fd4813 pa 0x0000000087f52000
.. .. ..0x0000003fffffe000: pte 0x0000000021fd00c7 pa 0x0000000087f40000
.. .. ..0x0000003ffffff000: pte 0x0000000020001c4b pa 0x0000000080007000
```

The first line displays the argument to `vmprint`. After that there is a line for each PTE, including PTEs that refer to page-table pages deeper in the tree. Each PTE line is indented by a number of `" .."` that indicates its depth in the tree. Each PTE line shows its virtual addresss, the pte bits, and the physical address extracted from the PTE. Don't print PTEs that are not valid. In the above example, the top-level page-table page has mappings for entries 0 and 255. The next level down for entry 0 has only index 0 mapped, and the bottom-level for that index 0 has a few entries mapped.
Your code might emit different physical addresses than those shown above. The number of entries and the virtual addresses should be the same.

Hints

- Use the macros at the end of the file `kernel/riscv.h`.
- The function `freewalk` may be inspirational.
- Use %p in your `printf` calls to print out full 64-bit hex PTEs and addresses as shown in the example.

5 Superpages

The RISC-V paging hardware supports two-megabyte pages as well as ordinary 4096-byte pages. The general idea of larger pages is called superpages, and (since RISC-V supports more than one size) 2M pages are called megapages. The operating system creates a superpage by setting the PTE_V and PTE_R bits in the level-1 PTE, and setting the physical page number to point to the start of a two-megabyte region of physical memory. This physical address must be two-mega-byte aligned (i.e., a multiple of two megabytes). You can read about this in the RISC-V privileged manual by searching for megapage and superpage; in particular, the top of page 112. Use of superpages decreases the amount of physical memory used by the page table, and can decrease misses in the TLB cache. For some programs this leads to large increases in performance.
Your job is to modify the xv6 kernel to use superpages. In particular, if a user program calls `sbrk()` requesting **2 MB or more** of memory, the kernel should identify every **2 MB-aligned** subregion within the newly allocated address range. Each such subregion should be mapped using a single superpage, while the remaining unaligned portions of the allocation should continue to use regular **4 KB** pages. A correct implementation is expected to pass the `superpg_fork` and `superpg_free` test cases when running `pgtbltest`.

Examples

The following examples illustrate the expected behavior of superpage allocation for different initial process sizes.

1. **No superpage-aligned region**

   Initial process size: **4 KB**

   ```
   sbrk(2101248);    // 2 MB + 4 KB
   ```

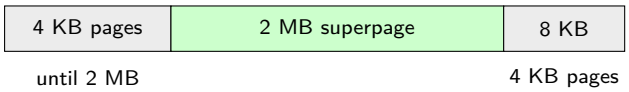
   Since the newly allocated region begins at virtual address 4 KB, it contains no 2 MB-aligned subregion of size 2 MB. The entire allocation must therefore use regular 4 KB pages.

2. **One aligned superpage with rest as regular pages**

   Initial process size: **4 KB**

   ```
   sbrk(4198400);    // 4 MB + 4 KB
   ```

   The allocator should first map regular 4 KB pages until the process reaches the first 2 MB-aligned virtual address. It should then allocate one 2 MB superpage, and finally map the remaining 8 KB using regular 4 KB pages.



Hints

- Read `superpg_fork` and `superpg_free` in `user/pgtbltest.c`.
- A good place to start is `sys_sbrk` in `kernel/sysproc.c`, which is invoked by the `sbrk` system call. Follow the code path to the `growproc` function that eagerly allocates memory for `sbrk`.
- Your kernel must be able to allocate and free two-megabyte physical memory regions. You must modify `kalloc.c` to reserve a small number of two-megabyte physical regions and implement the `superalloc()` and `superfree()` functions, inside the `#ifdef`'s, i.e where "`//YOUR CODE HERE - needed for the superpage lab`" is mentioned. For the purposes of this assignment, supporting up to 32 superpages is sufficient.**The allocator you write here shall be used across the tests for superpages**.
- Superpages must be allocated when a process with superpages forks, and freed when it exits; you'll need to modify `uvmcopy()` and `uvmunmap()`.
- When `sbrk` frees a superpage partially (e.g., freeing the last 4096 bytes of a superpage), you will need to "demote" a super page into regular pages.

Real operating systems dynamically promote a collection of pages to a superpage, something similar to what we will do with our Buddy Allocator in the next part of the assignment. The following reference explains why that is a good idea and what is hard in a more serious design: Juan Navarro, Sitaram Iyer, Peter Druschel, and Alan Cox, *Practical, Transparent Operating System Support for Superpages*, SIGOPS Oper. Syst. Rev., 36(SI):89–104, December 2002.

https://www.usenix.org/conference/osdi-02/practical-transparent-operating-system-support-superpages
This reference summarizes superpage implementations for different operating systems:
Weixi Zhu, *A Comprehensive Analysis of Superpage Management Mechanism and Policies*, USENIX Annual Technical Conference (ATC 2020).
https://www.usenix.org/conference/atc20/presentation/zhu-weixi

6 Buddy Allocator

In this part of the lab, you will replace xv6's physical page allocator with a **buddy allocator**. The original xv6 allocator manages memory one 4 KB page at a time. While the minimum size of a memory block allocated by our buddy allocator is also 4KB, it instead manages contiguous physical memory in blocks with sizes as powers-of-two number of pages, making it possible to efficiently allocate large contiguous regions while limiting external fragmentation. The allocator you build here may be used by the superpage code in the previous part of the assignment for your own testing purposes.

6.1 Background

The allocator manages physical memory using *blocks* of different *orders*. An order-k block beginning at physical address `pa` is the contiguous physical memory interval

$$[\texttt{pa},\ \texttt{pa} + 2^k \cdot \texttt{PGSIZE}),$$

where it `pa` is aligned to $2^k \cdot$ `PGSIZE`. An order-k block therefore contains 2^k contiguous pages, with the largest block in this lab being 2 MB (512 pages).
When a contiguous allocation request cannot be satisfied by a free block of the requested order, the allocator recursively splits the smallest available larger block into two equal-sized halves. These two halves are called buddies. Splitting continues until a free block of the requested order is obtained.
The reverse operation is called coalescing. Whenever two buddies are both free and have the same order, they are to be merged into their parent block. Coalescing may continue recursively toward larger orders, allowing large contiguous regions to be reconstructed after smaller allocations have been freed.

6.2 Provided Code

The file `kernel/buddyalloc.c` contains the allocator infrastructure, including the metadata representation, free-list implementation, helper routines, and allocator initialization.
The buddy allocator is disabled by default. To build and test your implementation, remove the following definition in `kernel/buddyalloc.h`:

```
#define BUDDY_OFF
```

You may introduce additional private helper functions or modify the existing infrastructure, but public interfaces should not be modified and all your code should reside within buddyalloc.c.

6.3 Metadata Representation

For each page-aligned physical address, we store one byte of metadata containing a **status** and an **order**. Recall that any page-aligned address that satisfies the required alignment is the

starting address of an order-k block for every valid k. However, at any given instant, only one of these blocks, if any, is represented by the allocator as an *independent block*: a block that is either free (and therefore present on a freelist) or allocated. It may be seen that a physical address is not permanently associated with a single order. As the allocator performs splits and coalescing, the same address may successively become the head of an order-8 independent block, an order-7 independent block, and later return to a larger order. The metadata byte records which of these it currently represents.
The **status** is one of the following:

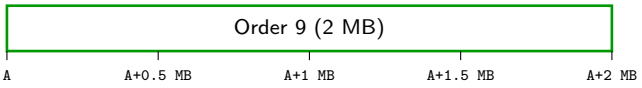
- `FREE`: the address is currently the head of an independent block that is available for allocation.
- `ALLOCATED`: the address is currently the head of an independent block that has been allocated.
- `SUBSUMED`: the address is currently not the head of an independent block but lies inside a larger, unsplit block.

The **order** is interpreted accordingly. For `FREE` and `ALLOCATED`, it is the order of the independent block currently beginning at that address. For `SUBSUMED`, there is no current independent block to describe. Instead, the stored order records the order of the *future* independent block that this address would be the head of if its immediate enclosing block were split.
Your implementation must ensure that the metadata and free lists remain consistent after every allocation, split, and free operation.

Example

Initially, physical memory is partitioned into a collection of free 2 MB independent blocks (order-9 blocks). A process now requests 0.5 MB (128 pages), corresponding to an order-7 block. Since the smallest available block of order at least 7 is an order-9 block, let the chosen independent block for splitting begin at physical address `A`. The allocator recursively splits this block until an order-7 block is obtained. We consider a valid execution which repeatedly splits the left child.

Initial state



Only the order-9 block is an independent block. The other three addresses already have metadata entries, but they are marked `SUBSUMED`: each lies inside the order-9 block, and none of them is currently the head of an independent block on its own.

Physical address	Order	Status
`A`	9	`FREE`
`A + 0.5 MB`	7	`SUBSUMED`
`A + 1 MB`	8	`SUBSUMED`
`A + 1.5 MB`	7	`SUBSUMED`

The stored order at a SUBSUMED address is the order of its *future* independent block it would be the head of the instant its immediate enclosing block is split. For example, A + 1 MB's immediate enclosing block is the order-9 block at A; splitting that block once makes A + 1 MB the head of an independent order-8 block, so its future order is 8. Likewise, A + 0.5 MB's immediate enclosing block would, after the order-9 split, be the order-8 block at A; splitting that block would make A + 0.5 MB the head of an independent order-7 block, so its future order is 7-even though, at this initial moment, the order-8 block containing it has not become independent yet.

After `buddyalloc(7)` Assume the allocator splits the order-9 block into two order-8 buddies, splits the left order-8 block once more, and allocates the left order-7 block.



The metadata is now updated as follows:

Physical address	Order	Status
A	7	ALLOCATED
A + 0.5 MB	7	FREE
A + 1 MB	8	FREE
A + 1.5 MB	7	SUBSUMED

Two of the addresses that were previously SUBSUMED are now the head of independent blocks. Splitting the order-9 block turns A + 1 MB into the head of a free order-8 block, and splitting that order-8 block in turn makes A and A + 0.5 MB the heads of two independent order-7 blocks, one allocated and one free.
A + 1.5 MB, however, is unaffected. Its immediate enclosing block is the order-8 block beginning at A + 1 MB, and since that block is never split, A + 1.5 MB remains SUBSUMED with the same future order, 7. Its metadata entry is unchanged because the split that would make it an independent block never occurs.

6.4 Your Task

Implement the following two public functions:

```
void *buddyalloc(uint8 order);
void  buddyfree(void *pa, int npages);
```

- `buddyalloc(order)` attempts to allocate one contiguous physical block containing 2^{order} pages. If one or more free blocks of exactly the requested order exist, your allocator may return any one of them. Otherwise, it must recursively split a free block of the smallest order greater than the requested order until a block of the requested order is obtained. At each split, either of the two buddy halves may be chosen for further splitting or eventual allocation; both choices are equally valid. If no free block of order $\geq$ the requested order exists, `buddyalloc()` must return 0.

- `buddyfree(pa, npages)` releases `npages` contiguous pages beginning at `pa`. The range being freed need not itself be a block previously returned by the buddy allocator; it may instead be a properly aligned sub-block of a larger allocated block. After freeing a block, the allocator must coalesce it with its buddy whenever the buddy is also free and of the same order, continuing recursively toward larger orders.

  The following preconditions must hold:

  - `npages` is a supported power of two;
  - `pa` is aligned to `npages * PGSIZE`;
  - the range `[pa, pa + npages * PGSIZE)` lies entirely within a single block currently allocated by the buddy allocator.

  It is to be noted that although general allocators do not explicitly require the number of pages while freeing, we are following a more explicit API.

- The buddy relationship is determined entirely by the recursive splitting of an initially larger block. Whenever an order-$(k+1)$ block is split, the two resulting order-k blocks become buddies of each other. For any block of fixed size and starting physical address, its buddy is fixed throughout the lifetime of the allocator.

  So, you must **not** compact or move allocated memory in order to create larger free blocks. The allocator is only allowed to split existing blocks and coalesce existing buddy pairs.

  For example, consider two order-$(k+1)$ blocks that are themselves buddies, and suppose each is split into two order-k blocks. This produces four contiguous order-k blocks. Even if the second block from the first parent and the first block from the second parent are both free, they are *not* buddies-they originated from different parent blocks. Therefore, they must not be merged into an order-$(k+1)$ block, and the allocator must not move or rearrange the allocated blocks to make such a merge possible.

6.5 Testing

A correct implementation should pass `buddy_contig` , `buddy_coalescing` in `buddytest.c` and the xv6 regression suite `usertests -q`. **All the tests for this part of the assignment, both public and private shall be run by removing the `#define BUDDY_OFF`.**

6.6 References

The following references provide additional background on buddy allocation to help with the lab.

1. Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, Chapter 17: Free-Space Management.
   OSTEP – Free-Space Management (Buddy Allocation)

2. Redha Gouicem and Julien Sopena, *Linux Kernel Programming*, Buddy Allocator and Linux Implementation.
   RWTH Aachen – Linux Buddy Allocator Lecture

3. Donald E. Knuth, *The Art of Computer Programming, Volume 1: Fundamental Algorithms*, 3rd Edition, Section 2.5.
   The Art of Computer Programming (TAOCP)

7 Submission

7.1 Time Spent

Create a new file, `time.txt`, and put in it a single integer—the number of hours you spent on the lab.

7.2 Answers

Write your answers to the questions for the first part of the assignment in `answers-pgtbl.txt`.

7.3 References

Mention all sources that you referred to while working on this lab assignment. If you collaborated with other students, clearly mention their names and describe the specific parts of the assignment on which you collaborated.
If you used Generative AI for any part of the assignment-whether to understand concepts, debug code, or generate a portion of the solution-you must clearly state **which part** it was used for and include the corresponding prompt logs.

7.4 Submit

Assignment submissions are handled via Moodle. When you are ready to submit, run:

```
$ make zipball
```

which generates `lab.zip`. If you have either uncommitted changes or untracked files, you will see output similar to the following:

```
M hello.c
?? bar.c
?? foo.pyc
Untracked files will not be handed in. Continue? [y/N]
```

Inspect the above lines and make sure all files that your lab solution needs are tracked, i.e., not listed on a line that begins with `??`. You can cause Git to track a new file using `git add {filename}`.

- Please run `make grade` to ensure that your code passes all of the tests. The autograder will use the same grading program possibly with additional test cases to assign your submission a grade.
- Commit any modified source code before running `make zipball`.
- Rename the resulting `lab.zip` to `<ROLL_NUMBER>_OS_LAB_3_PageTables.zip` and upload it to Moodle.

8 Optional Challenge Exercises

- Implement some ideas from the paper referenced above to make your superpage design more practical.
- Unmap the first page of a user process so that dereferencing a null pointer results in a page fault. You will have to modify `user.ld` to start the user text segment at, for example, 4096 instead of 0.

- Add a system call that reports dirty (modified) pages using the `PTE_D` bit.
- Implement Address Space Layout Randomisation (ASLR) for xv6.

Acknowledgement

A significant portion of this lab assignment is derived from the MIT 6.1810/6.828 Operating Systems Engineering labs:
https://pdos.csail.mit.edu/6.1810/2025/labs/pgtbl.html
The original xv6 teaching materials and lab framework were developed by the MIT PDOS group and have been adapted for use in the CS3500 Operating Systems course.